SCCAD 3nm PDK

User Manual

OpenROAD Edition

Version	v1.0
Release Date	TBD
Organization	SCCAD Lab, University of Southern California
Tool	OpenROAD 26Q1 (open-source)
Contact	sw.jung@gatech.edu (Sungwoo Jung)

This manual provides comprehensive guidance for users and developers working with the SCCAD 3nm Process Design Kit (PDK). It covers PDK installation from GitHub, environment configuration, and step-by-step instructions for running Place-and-Route (PnR) using OpenROAD — an open-source RTL-to-GDS tool chain.

1 Introduction

1.1 Overview of SCCAD 3nm PDK

The SCCAD 3nm Process Design Kit (PDK) is developed and maintained by the SCCAD Laboratory to support academic and research-oriented digital IC design flows at the 3 nm technology node. The PDK provides a complete set of design enablement files including technology files, standard-cell libraries, SPICE models, and interconnect models. This allow designers to perform synthesis, place-and-route (PnR), and sign-off verification using industry-standard EDA tools.

Several open-source 3 nm PDKs are available to the research community. The table below positions SCCAD-3nm alongside two other academic PDKs to highlight the distinguishing capabilities of this release.

Feature	Academic PDK A	Academic PDK B	SCCAD-3nm
Tech Node	3nm	3nm	3nm
Device Type	GAAFET	GAAFET	GAAFET
Cell Heights	5.5T	6T	6T, 5T
Power Rail	FSPR	BPR	FSPR, BPR
Back-side Metal	No	No	Yes

The SCCAD-3nm PDK offers several capabilities that extend the design space available to researchers:

- **Dual cell-height support (6T and 5T).** Providing both 6-track and 5-track standard-cell libraries gives designers flexibility to trade off area density against routability.
- **Both Front-side and Buried Power Rail (FSPR & BPR).** Supporting both power delivery architectures allows direct comparison of conventional front-side routing against buried power rail approaches.
- **Back-side metal support.** SCCAD-3nm includes back-side metal interconnect, enabling exploration of back-side power delivery network (BS-PDN) and signal routing techniques.

1.2 Key Features and Technology Highlights

Technology Node: 3 nm GAAFET

The SCCAD-3nm PDK is built on a 3 nm Gate-All-Around FET (GAAFET) process. Unlike FinFET, where the gate wraps around three sides of a fin, GAAFET surrounds the channel on all four sides using stacked nanosheet or nanowire structures. This architecture provides superior electrostatic control, reduced short-channel effects, and improved drive current at scaled supply voltages.

Table 1 summarizes the key device physical parameters of the SCCAD-3nm NSFET device in comparison with a representative 5 nm FinFET process.

Table 1. Comparison of device physical parameters of FinFET and NSFET device.

Physical Parameters (nm)	TSMC 5nm	NSFET 3nm
CPP	48	42
Fin/NS pitch	25	NA
Gate length	15	12
Spacer length	6	5
# Fin/NS	2	3
Fin/NS thickness	6	5
Fin/NS width	NA	30, 20
Fin/NS stack height	55	55

Standard-Cell Library

The SCCAD-3nm PDK includes two standard-cell libraries. The 6-Track (6T) library uses a Front-side Power Rail (FSPR) architecture (cell height 144 nm, M1 power rails), and is the library supported by this OpenROAD. The complete cell set comprises 57 standard cells.

Table 2. Standard-cell library contents.

STD Cells	Drive Strength Variants
INV	x1, x2, x3, x4, x5, x6, x7, x8, x10, x12, x14, x16
BUF	x1, x2, x3, x4, x5, x6, x7, x8, x10
NAND, NOR	2x1, 3x1, 4x1, 2x2, 3x2
AND, OR	2x1, 3x1, 4x1
XOR, XNOR	2x1, 3x1
AOI, OAI	(21, 22, 221, 222, 311, 31) x1
MUX	2x1
D Flip-Flop	Hx1 (async.), HQNx1 (sync.)
DHL (latch)	x1
Total	57 cells

Back-end-of-Line (BEOL) Technology

The SCCAD-3nm PDK features a comprehensive BEOL stack spanning from back-side metal layers through front-side routing layers (MB2–M9, VB1–V8). Table 3 summarizes the key physical parameters of each layer used in the OpenROAD flow.

Table 3. BEOL Metal and Via Layers — Physical Parameters.

Tier	Metal Layer	W (nm)	Resistance (Ω/□)

Back-side	MB2–MB1	61	11
BPR layer	MBPR	25	65
Front-Side	M1–M3	12	347
Front-Side	M4, M5	18	101
Front-Side	M6, M7	24	44
Front-Side	M8, M9	32	29

2 Getting Started — PDK Installation from GitHub

2.1 Cloning the Repository

The SCCAD 3nm PDK is hosted on GitHub. Use the following commands to clone the repository to your local workstation or server.

HTTPS:

```
git clone https://github.com/sjung366/SCCAD-3nm-PDK.git
cd SCCAD-3nm-PDK
```

SSH (recommended for development):

```
git clone git@github.com:SCCAD-Lab/SCCAD-3nm-PDK.git
cd SCCAD-3nm-PDK
```

If the repository uses Git submodules, initialize them as follows:

```
git submodule update --init --recursive
```

2.2 Environment Setup

Before running any EDA tool, the following environment variables must be configured manually. These variables define the paths to key PDK components and must be set in your shell environment (e.g., .bashrc or .cshrc) or exported at the beginning of each session.

Required Environment Variables:

```
# Root directory of the PDK repository
export PDK_ROOT=/path/to/sccad-3nm-pdk

# Technology LEF path (OpenROAD)
export TECH_LEF=$PDK_ROOT/TECH-LEF

# Cell LEF path
export LEF_PATH=$PDK_ROOT/LEF
```

```
# Liberty timing library path
export LIB_PATH=$PDK_ROOT/LIB_DB

# RC path (for OpenRCX calibration)
export RC_PATH=$PDK_ROOT/RC

# OpenROAD installation path
export OPENROAD_BIN=/path/to/OpenROAD-flow-scripts/tools/install/OpenROAD/bin/openroad
```

Verify that the key PDK files are accessible:

```
echo $PDK_ROOT
ls $TECH_LEF
ls $LEF_PATH
ls $LIB_PATH
```

3 Running PnR with OpenROAD

3.1 Essential PDK Files for PnR

Before launching a PnR run, it is important to understand which PDK files are loaded at each stage of the flow and what role they play. Unlike Synopsys ICC2 (which uses the NDM format) or Cadence Innovus (which uses QRC tech files), OpenROAD loads technology LEF, cell LEF, and Liberty files directly. RC models are provided through setRC.tcl for placement-stage estimation and through the OpenRCX rules file for post-route extraction.

Table 5. Essential PDK files for OpenROAD PnR.

File	Format	Description
`3nm_GAA_FSPR.tech.lef`	`.lef`	Technology LEF: routing layers, pitches, via rules, manufacturing grid, site definition
`3nm_GAA_FSPR.lef`	`.lef`	Cell LEF: standard cell abstracts — pin locations, cell dimensions, obstruction layers
`3nm_GAA_FSPR_lvt_nldm.lib`	`.lib`	Liberty: timing arcs, power models, input/output capacitance per cell pin
`netlist.v`	`.v`	Gate-level Verilog netlist from synthesis
`constraint.sdc`	`.sdc`	Timing constraints: clock definition, I/O delays
`3nm.tracks`	`.tcl`	Routing track grid: make_tracks commands per metal layer
`3nm.pdn.tcl`	`.tcl`	Power delivery network strategy: M1 follow-pin VDD/VSS rails
`setRC.tcl`	`.tcl`	Per-layer wire R (kΩ/μm) and C (pF/μm) for placement-stage parasitic estimation

`rcx_patterns.rules`	`.rules`	StarRC-calibrated OpenRCX R/C tables for post-route parasitic extraction

> **NOTE:** *The setRC.tcl and rcx_patterns.rules files are specific to OpenROAD and have no equivalent in the ICC2 or Innovus PDK directories. They must be generated separately.*

3.2 PnR Flow Overview

The SCCAD-3nm PDK supports a complete PnR back-end flow using OpenROAD. The flow is driven by a Tcl script (ecg_3nm.tcl) that sets design parameters and calls flow.tcl for all physical implementation stages. The overall flow spans from design setup through detailed routing, parasitic extraction, and final PPA reporting.

Table 6 summarizes the complete PnR flow stages in execution order.

#	Stage	Key Command	Output / Checkpoint
0	Read Libraries	`read_lef / read_liberty / read_verilog / read_sdc`	61K instances loaded into database
1	Floorplan + Tracks	`initialize_floorplan / source 3nm.tracks`	597 rows × 2047 sites created
2	Buffer Removal	`remove_buffers`	Synthesis buffers cleaned (833 removed)
3a	IO Placement (1st pass)	`place_pins -hor_layers M2 -ver_layers M3`	Pin locations set for global placement
4	PDN Generation	`source 3nm.pdn.tcl / pdngen`	M1 follow-pin VDD/VSS rails generated
5	Global Placement	`global_placement -routability_driven`	Cells placed (~66% utilization)
3b	IO Placement (2nd pass)	`place_pins`	Pins re-optimized post-placement
6	Pre-CTS Timing Repair	`repair_design (uses setRC.tcl)`	Slew/cap violations repaired
7	Clock Tree Synthesis	`clock_tree_synthesis -root_buf BUFx4`	CTS complete, cts.db saved
8	Post-CTS Timing Repair	`repair_timing (uses setRC.tcl)`	Setup + hold violations repaired
9	Detailed Placement	`detailed_placement`	All cells legalized to row grid
10	Pin Access	`pin_access`	Via access patterns computed
11	Global Routing	`global_route -congestion_iterations 100`	route_guide file generated
12	Antenna Repair	`repair_antennas`	(Skipped: no antenna diode in PDK)
13	Detailed Routing	`detailed_route`	DRV = 0, route.db / route.def saved
14	RC Extraction	`extract_parasitics -ext_model_file rcx_patterns.rules`	SPEF generated from actual wire geometry

15	Final Report	`report_checks / report_power / report_design_area`	WNS, TNS, power, area reported

3.3 Step-by-Step PnR Commands

Launching a PnR Run

OpenROAD is executed inside a Docker container using the provided run script. The run_openroad.sh script handles volume mounting, library installation, and background execution automatically.

Basic usage:

```
cd /path/to/openroad
./run_openroad.sh sweep/my_experiment ecg_3nm.tcl
```

With thread count control (recommended on shared servers):

```
./run_openroad.sh sweep/my_experiment ecg_3nm.tcl -threads 16
```

The script outputs the log file path and monitoring commands:

```
Starting OpenROAD: ecg_3nm.tcl (threads: 16)
Work dir: /path/to/sweep/my_experiment
Log: /path/to/sweep/my_experiment/run.log
Running in background (PID 12345)
Monitor:  tail -f /path/to/sweep/my_experiment/run.log
```

Monitoring Progress

To monitor the run in real time:

```
# Live log tail
tail -f sweep/my_experiment/run.log

# Check current PnR stage
grep '^\[INFO [A-Z]' sweep/my_experiment/run.log | tail -5

# Check CPU and memory usage
docker stats --no-stream
```

The stage prefix codes in the log indicate which module is active:

Prefix	Stage
`IFP`	Floorplan
`GPL`	Global placement
`RSZ`	Resize / timing repair
`CTS`	Clock tree synthesis

DPL	Detailed placement
GRT	Global routing
DRT	Detailed routing

Checking Results

After the run completes (log ends with openroad>), use the following commands to extract key PPA metrics:

```
LOG=sweep/my_experiment/run.log

# Final timing (Setup WNS / Hold WNS / TNS)
grep 'worst slack\|tns max' $LOG | tail -6

# Clock skew
grep 'setup skew' $LOG | tail -2

# DRV and antenna violations (must be 0)
grep 'ANT-0001\|ANT-0002' $LOG | tail -4

# Area and utilization
grep 'Design area' $LOG | tail -1

# Power
grep '^Total' $LOG | tail -3
```

Viewing Results in OpenROAD GUI

The OpenROAD GUI can be launched using the provided script, optionally loading a checkpoint database directly:

```
# Open empty GUI
./open_openroad_gui.sh

# Load routing result directly
./open_openroad_gui.sh sweep/my_experiment/results/ecg_3nm-tcl_route.db
```

Inside the GUI, navigate to the mounted volume path (/host_openroad/sweep/...) to locate result files. The GUI provides heat maps for routing congestion, timing path highlighting, and DRC violation viewing.

Running Multiple Experiments in Parallel

Multiple PnR runs can be launched simultaneously. Create independent experiment directories first:

```
# Create experiment directories
cp -r sweep/baseline sweep/exp_density70
cp -r sweep/baseline sweep/exp_density80
```

```
# Edit parameters in each directory
vim sweep/exp_density70/3nm/3nm.vars   # set global_place_density 0.70
vim sweep/exp_density80/3nm/3nm.vars   # set global_place_density 0.80

# Launch all simultaneously
./run_openroad.sh sweep/exp_density70 ecg_3nm.tcl -threads 16 &
./run_openroad.sh sweep/exp_density80 ecg_3nm.tcl -threads 16 &
```

> **NOTE:** *Each experiment directory must be self-contained with its own copies of ecg_3nm.tcl, ecg_3nm.sdc, ecg_3nm.v, flow.tcl, flow_helpers.tcl, helpers.tcl, and the 3nm/ PDK directory.*

References

[1] J.-S. Yoon, J. Jeong, S. Lee, J. Lee, S. Lee, R.-H. Baek, and S. K. Lim, "Performance, Power, and Area of Standard Cells in Sub 3 nm Node Using Buried Power Rail," *IEEE Transactions on Electron Devices*, vol. 69, no. 3, pp. 894–899, Mar. 2022. doi: 10.1109/TED.2021.3138865

[2] S. M. Shaji, L. Zhu, J. Yoon, and S. K. Lim, "A Comparative Study on Front-Side, Buried and Back-Side Power Rail Topologies in 3nm Technology Node," in *Proc. IEEE/ACM Int. Symp. Low Power Electron. Design (ISLPED)*, 2023. doi: 10.1109/ISLPED58423.2023.10244504

[3] L. T. Clark, V. Vashishtha, L. Shifren, A. Gujja, S. Sinha, B. Cline, C. Ramamurthy, and G. Yeric, “ASAP7: A 7-nm FinFET Predictive Process Design Kit,” *Microelectronics Journal*, vol. 53, pp. 105–115, Jul. 2016, doi: 10.1016/j.mejo.2016.04.006.

[4] S. Sadangi, V. Pasumarthy, S. Pitts, and W. R. Davis, “FreePDK3: A Novel PDK for Physical Verification at the 3nm Node,” Master’s Thesis, NC State University, 2021.

[5] D. E. Shim, P. Kumar, A. A. Kini, M. Mallikarjuna, M. N. H. Shazon, and A. Naeemi, “GT3: An Open-Source 3-nm GAAFET PDK and Platform for End-to-End Evaluation of Emerging Technologies,” *IEEE Transactions on Electron Devices*, doi: 10.1109/TED.2025.3540760, 2025.